SE411 / FALL 2026-27 / PART 1

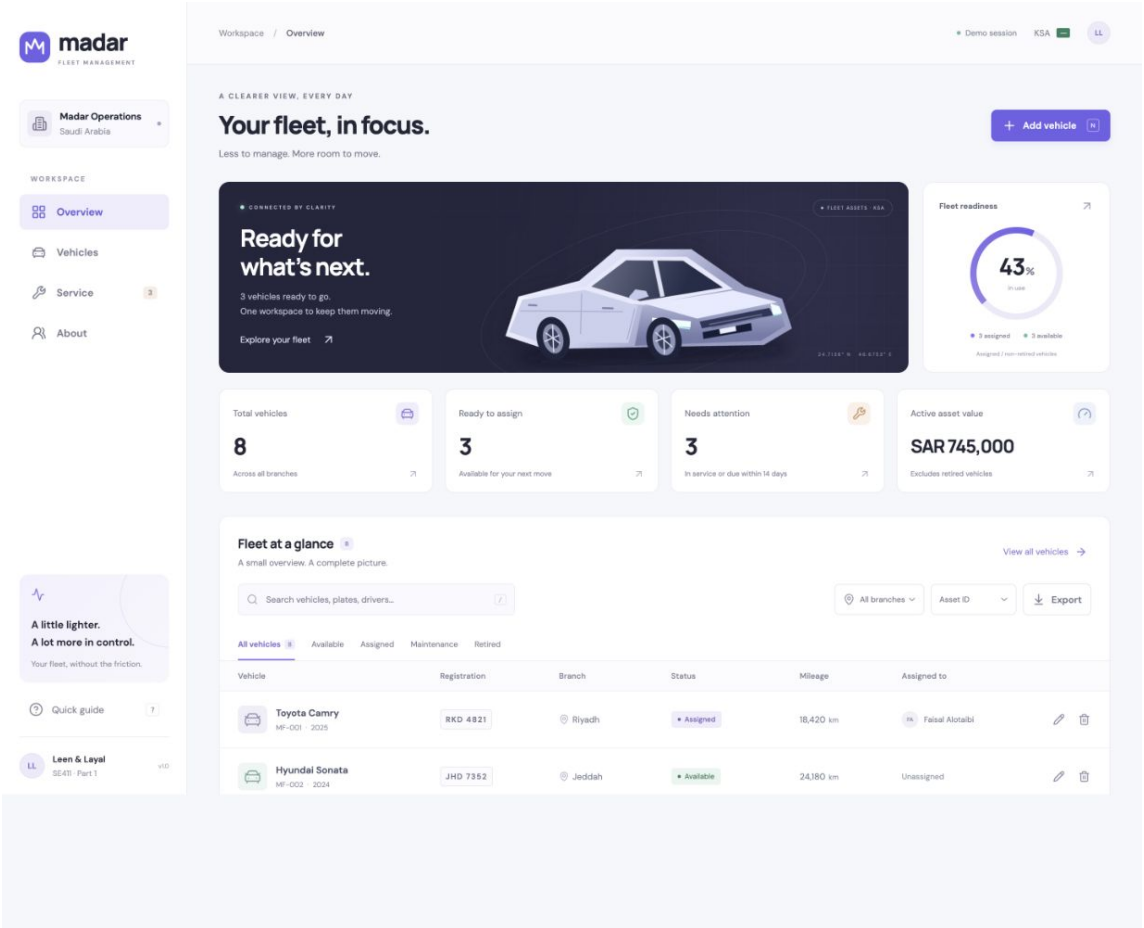
# Madar Fleet

## Your fleet, in focus.

Company vehicle asset management
React frontend project report


**Leen Alshaya**
221410456

**Layal Alhashim**
220511246


Prepared 22 October 2026 | Saudi business setting
Fictional sample records. Session-only object collection.

# 1. Project overview

Madar Fleet is a frontend application for managing one category of company asset: vehicles. It presents a fictional Saudi company with branches in Riyadh, Jeddah, Dammam, and Al Khobar. A fleet coordinator can register vehicles, inspect their details, assign drivers, update condition-related status and mileage, and remove records.

The central design goal is to accomplish common tasks in fewer gestures. Summary cards link directly to useful views; selecting a vehicle opens an edit drawer; filters combine in a single view; keyboard shortcuts support frequent operations. The interface uses restrained violet accents, generous whitespace, a dark overview panel, and an original vehicle illustration for a slightly futuristic appearance.

# 2. Objectives and scope

1. Implement a React application with a navigable menu.

2. Support create, read, update, and delete operations on a collection of vehicle objects.

3. Present both provided team members' names and student IDs on an About page.

4. Supply architecture, technology, feature, requirement, and user documentation.

5. Generate and execute tests with AI assistance.

6. Prepare a repository suitable for public GitHub submission.

Part 1 is entirely client-side. It has no database, backend API, authentication system, or cross-user synchronization. Sample data is fictional. Part 2 is reserved for the manually coded backend, in accordance with the assignment.

# 3. Intended user and workflows

The intended user is a fleet coordinator in a Saudi business. The coordinator needs to see which vehicles are available, find a vehicle by its asset ID or registration, assign a driver, and identify upcoming maintenance.

**Register:** select Add vehicle or press N; enter the manufacturer, model, year, registration, branch, status, mileage, and value; optionally enter the next service date and notes; save. The application validates the record and allocates a session-unique asset ID.

**Inspect and update:** search, filter, or sort the registry; select a vehicle name or its edit icon; update the required details in one panel; save or cancel.

**Remove:** select the delete icon; inspect the named vehicle in the confirmation dialog; confirm deletion. An immediate Undo action restores the deleted record while its notification is present.

**Service planning:** select Service or the Needs attention summary. The queue contains non-retired vehicles in Maintenance, plus non-retired vehicles with a next-service date on or before today + 14 days. An overdue date remains in the queue. Selecting a record allows the coordinator to change its date or status.

# 4. Technology stack

| Layer | Technology | Purpose |
|---|---|---|
| UI | React / React DOM 19.3.1 | Components, state, rendering |
| Build | Vite 7.3.7; React plugin 5.2.0 | Development server and static production build |
| Language | JavaScript ES modules, JSX | Application and domain logic |
| Styling | CSS | Responsive layout, focus states, reduced motion |
| Icons | Lucide React 0.468.0 | Consistent navigation and action icons |
| Tests | Vitest 3.2.7 | Unit and interaction test runner |
| UI testing | Testing Library React 16.3.3; user-event 14.6.8; jest-dom 6.9.1 | User-oriented interaction assertions |
| Test environment | jsdom 26.1.0 | Browser-like DOM for automated tests |
| Coverage | Vitest V8 coverage 3.2.7 | Coverage instrumentation |
| Typography | DM Sans; Manrope | Self-hosted interface fonts |
| Continuous integration | GitHub Actions | Install, test, and build after repository upload |

Versions above describe the verified installation. `package-lock.json` is the source of truth for reproducible installation with `npm ci`. Node.js 22.12 or newer is required. The generated files in `dist/` can be served by a static web host; the app is not opened by double-clicking index.html.

# 5. Architecture

```
index.html
    -> src/main.jsx (React root + StrictMode)
        -> App.jsx (single source of truth: vehicles[])
            -> Overview / Vehicles / Service / About
            -> VehicleForm.jsx -> validation -> save callback
            -> Modal.jsx -> focus trap / Escape / restoration
            -> VehicleArt.jsx -> decorative SVG
            -> lib/fleet.js -> pure domain calculations
            -> data/vehicles.js -> initial object collection
```

`App.jsx` owns the `vehicles` array with `useState`. Create operations append an object, update operations replace the matching object with `map`, and delete operations remove the matching object with `filter`. These immutable updates trigger React rendering without mutating the original collection. A sequence reference prevents reusing deleted asset IDs during the current session.

`VehicleForm.jsx` owns temporary form values and error messages. It validates input against the current collection, then passes valid values to the save handler. Closing or cancelling the form leaves stored records unchanged.

`lib/fleet.js` separates business rules from presentation. It contains validation, normalization, date calculations, filtering, sorting, summary calculations, ID generation, and safe CSV export. These functions accept their inputs explicitly, which makes them independently testable and reusable in a later phase.

Navigation uses hash links (`#overview`, `#vehicles`, `#service`, `#about`) and a `hashchange` listener. Direct links and browser back/forward work without server routing rules or a routing dependency. Navigating through the menu resets view filters; ordinary browser hash navigation changes the page.

All summaries and filtered views are derived from the current record collection. No separate copies of totals are stored, preventing totals from becoming stale after CRUD operations.

## 6. Data dictionary

Each record represents one individual vehicle, not a model's stock quantity.

| Field | Type | Meaning / rule |
|---|---|---|
| id | string | Generated asset ID, such as MF-009; immutable in the form |
| make | string | Required manufacturer; trimmed |
| model | string | Required model; trimmed |
| year | number | Whole year from 1990 through next calendar year |
| plate | string | Required registration label, unique ignoring spaces, hyphens and case |
| branch | string | Riyadh, Jeddah, Dammam, or Al Khobar |
| status | string | Available, Assigned, Maintenance, or Retired |
| driver | string | Required when Assigned; cleared for other statuses |
| mileage | number | Non-negative safe whole number, kilometres |
| value | number | Non-negative value up to SAR 100,000,000 |
| serviceDate | string | Optional valid YYYY-MM-DD date |
| notes | string | Optional notes; form limits input to 500 characters |

The registration labels are fictional examples, not a claim of conformity with Saudi plate-format regulations. The app accepts Arabic text, but does not validate official Arabic/Latin plate equivalence. Asset values are illustrative values, not current market quotations.

## 7. Data lifetime and calculations

Eight sample records are initialized by `createDemoVehicles()`. Service dates are relative to the day the session starts, so the initial demo consistently includes an overdue vehicle and two upcoming service records. The calendar day refreshes in the application every minute. Date-only service comparisons use UTC midnight arithmetic to avoid daylight-saving differences between dates; the definition of today follows the browser's local calendar.

Changes live only in memory. Refreshing, closing, or reopening the page restores the sample collection; separate tabs do not share edits. No database, localStorage, or sessionStorage is used. CSV export is a download of the current view, not application persistence. Import is not included.

- **Total vehicles:** count of all records, including retired vehicles.

- **Ready to assign:** count of Available records.

- **Needs attention:** count meeting the service-queue rule.

- **Active asset value:** sum of values for all non-retired records.

- **Utilization:** Assigned records divided by all non-retired records, rounded to a whole percentage; an empty active fleet returns 0%.

For the initial sample, these values are 8 total, 3 available, 3 needing attention, SAR 745,000 active asset value, and 43% utilization.

## 8. Interface and accessibility

The desktop layout combines a fixed navigation rail, a compact workspace header, and a spacious task area. The overview contains fleet summaries; the registry offers search, branch filtering, status filters, sorting, and export. Editing stays in a single drawer. Mobile uses bottom navigation and vehicle cards with visible row actions.

The form uses visible labels, required markers, inline errors linked to their inputs, and focus on the first invalid field. Dialogs have accessible names, contain keyboard focus, close with Escape, and restore focus to the previous control. Icon-only actions have text labels for assistive technology. The app includes a skip link, current-page indication, status text alongside colour, a polite notification region, and reduced-motion styling.

Shortcuts: `/` focuses search, `N` opens Add vehicle, `?` opens Quick guide, and Escape closes the current dialog. Global shortcuts do not run while the user types into a form field. These measures improve accessibility; this submission does not claim a formal WCAG audit or exhaustive assistive-technology certification.

## 9. Validation and export safeguards

The form rejects missing required fields, duplicate registrations, invalid years, invalid branches/statuses, negative or fractional mileage, invalid monetary values, and impossible calendar dates. Assigned records require a driver. Retired vehicles remain visible as assets but are excluded from service attention and active-value/utilization calculations.

CSV output includes a UTF-8 byte-order mark, quoted fields, escaped double quotes, and formula neutralization for text starting with spreadsheet formula characters. It exports only the filtered view. React renders user input as text; no raw user HTML is inserted. There is no remote storage or sensitive production dataset.

## 10. Testing and evidence

The verified automated suite contains **81 passing checks** across two files: 60 domain tests and 21 application interaction tests. Executed commands were `npm run test:coverage` and `npm run build`. The separate test report records exact results, coverage scope, and manual browser checks. Test and build logs and a machine-readable coverage summary are included in `docs/evidence/`.

The tests exercise CRUD, duplicate registrations, assignment, combined filters, search, sorting, service boundaries, empty views, keyboard shortcuts, focus behavior, CSV export, and remount reset. Coverage is a supporting metric, not a guarantee that every visual state or browser is correct. Browser verification and

screenshots complement the automated DOM tests.

## 11. AI assistance

OpenAI Codex assisted with requirement interpretation, the Madar Fleet concept, visual design, React code, fictional sample data, test generation/execution, and documentation. The supplied brief, chosen category, Saudi setting, team identities, and design direction came from the user. No individual student contribution split has been invented. The students should review the implementation, understand the code, and accurately describe their own work during assessment.

## 12. Delivery and remaining administrative steps

The package contains source code, dependency lockfile, documentation, report, tests, evidence, screenshots, and a GitHub Actions workflow. GitHub upload is being performed by the user. The public repository URL is therefore not claimed or fabricated in this report. One student must submit the resulting public repository URL.

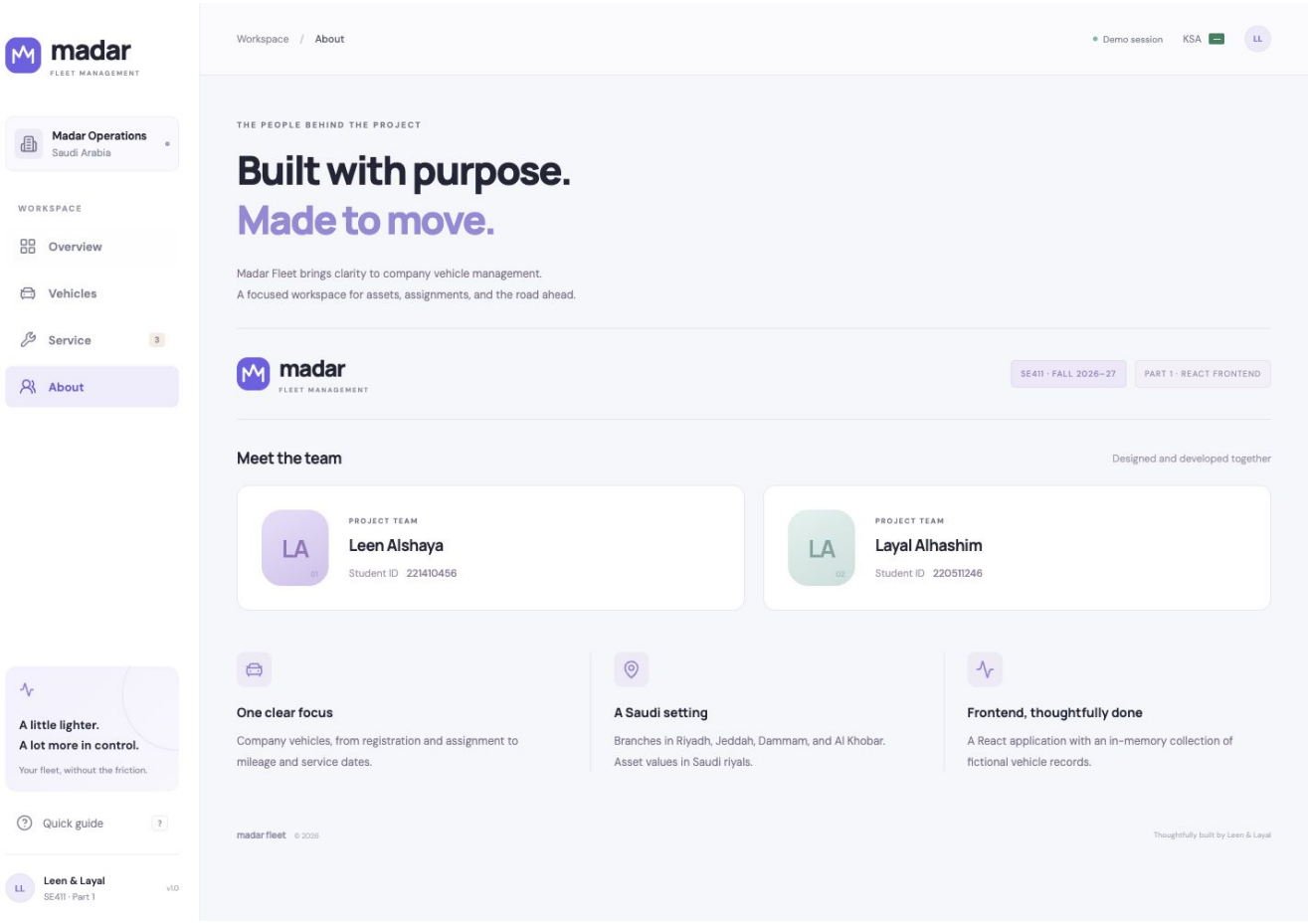
The assignment specifies teams of three to four, while only two actual identities were supplied. Confirm instructor approval for two members or add the remaining real members. The brief's printed deadline is 17 October 2026; this package was prepared on 22 October 2026. Any revised submission arrangement must be confirmed with the instructor.

## 13. Part 2 direction

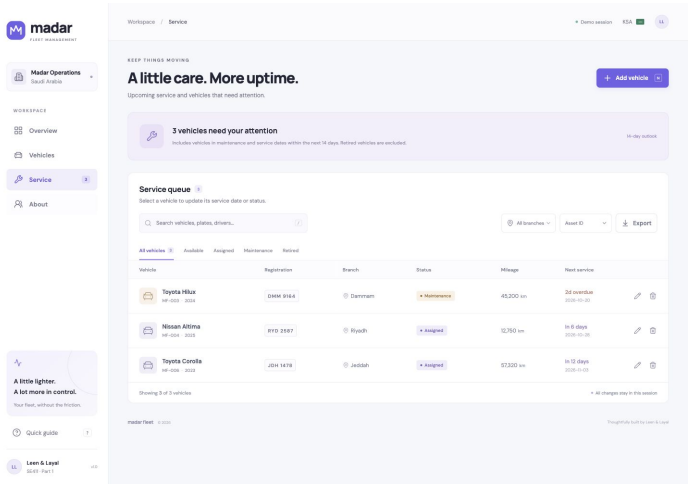
A future, manually written backend can provide database persistence, server-side validation, an API, authentication, and multi-user access. It should preserve the record shape and verify business rules server-side. Those capabilities are future work and are not part of the implemented Part 1 frontend.

# 14. Visual evidence

Screens captured from the production build. Desktop navigation, the registry, and team information use the same React state and component system as the mobile layout.



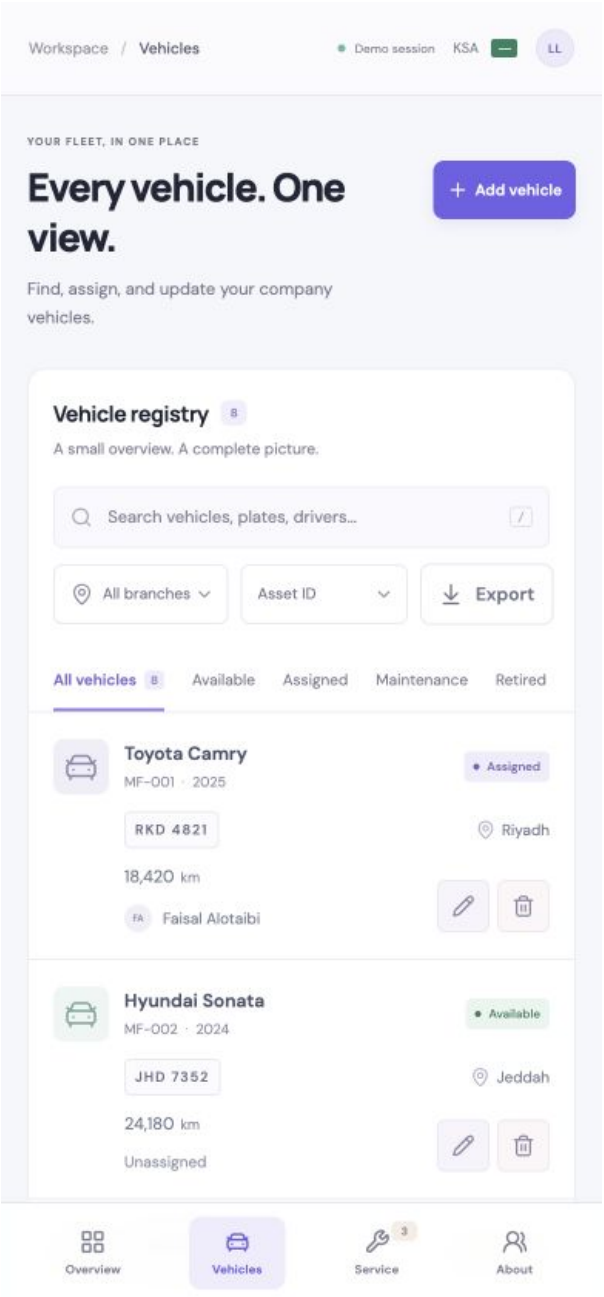

About page: both provided team members and exact student IDs.

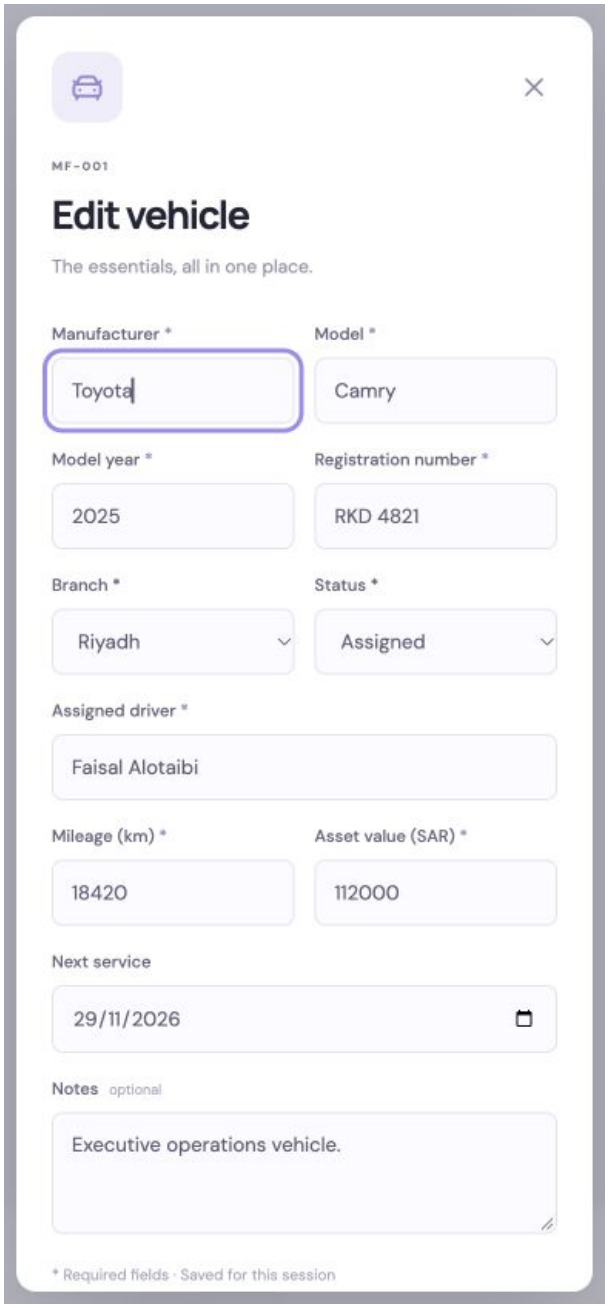



Service queue: maintenance and dates within the 14-day window.

## 15. Mobile experience

Vehicle cards retain direct edit and delete controls. Forms preserve labelled fields and use the available vertical space. The tested phone layout has no horizontal page overflow.




Left: responsive vehicle registry. Right: editing an existing assigned vehicle.

## Verification record

81 automated checks passed. The production build completed successfully. The detailed test report and raw execution evidence are included in the docs folder. GitHub upload and final course submission are user actions; no public repository URL or cloud CI result is claimed.